

JobHuntTracker System Technical Design

Autonomous Job Application Pipeline, JD Archival & Inbound Email Intelligence Engine

Architecture: C# .NET 9 / Blazor / Minimal API **Persistence:** EF Core + SQLite **LLM Engine:** Semantic Kernel / Structured JSON **Target:** Local / Self-Hosted

1. SYSTEM ARCHITECTURE & COMPONENT FLOW

1. Capture (Chrome Extension)

Manifest V3 extension parses current DOM text, metadata, URL, and salary indicators. Posts structured payload directly to local API endpoints.

2. Processing (.NET Core)

Hosted background worker connects via MailKit (IMAP) or Graph API, queries inbound threads, filters noise, and dispatches raw emails to LLM extractor.

3. Intelligence & Action

Structured outputs update DB states, correlate threads to companies, and dispatch Discord/Telegram/Desktop alerts for scheduling actions.

2. SOLUTION PROJECT STRUCTURE (.NET)

```
JobHuntTracker/
├── JobHuntTracker.sln
├── src/
│   ├── JobHuntTracker.Core/
│   │   ├── Models/
│   │   ├── Enums/
│   │   └── Interfaces/
│   ├── JobHuntTracker.Infrastructure/
│   │   ├── Data/
│   │   ├── Email/
│   │   ├── Extraction/
│   │   └── Notifications/
│   ├── JobHuntTracker.Api/
│   │   ├── Endpoints/
│   │   ├── Pages/
│   │   ├── Workers/
│   │   ├── Program.cs
│   │   └── appsettings.json
│   └── JobHuntTracker.Extension/
│       ├── manifest.json
│       ├── popup.html / popup.js
│       └── content.js
└── # Domain models, Enums, Interfaces, Value Objects
    # JobApplication, EmailInteraction, ApplicationNote
    # ApplicationStatus, InteractionType
    # IEmailPoller, IIntelligenceExtractor, INotifier
    # DB context, IMAP/Graph client, LLM client, Alerts
    # AppDbContext, SQLite Migrations, Configurations
    # ImapEmailPoller (MailKit), GmailClient
    # SemanticKernelParser / OpenAI Structured Parser
    # DiscordWebhookNotifier, ToastNotifier
    # Blazor Server UI + Minimal API Endpoints + Workers
    # Ingestion routes for browser extension
    # Kanban Board, Application Detail, JD Search
    # BackgroundServices for recurring email polling
    # Chrome Extension (Manifest V3)
```

3. CORE DATA STRUCTURES & MODELS

```
public enum ApplicationStatus
{
    Identified, Applied, Screening, TechnicalInterview, FinalRound, OfferReceived, Rejected, Withdrawn
}

public class JobApplication
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public string CompanyName { get; set; } = string.Empty;
    public string JobTitle { get; set; } = string.Empty;
    public string? JobUrl { get; set; }
    public string RawJobDescription { get; set; } = string.Empty;
    public string? SalaryRange { get; set; }
    public ApplicationStatus Status { get; set; } = ApplicationStatus.Identified;
    public DateTime DateCaptured { get; set; } = DateTime.UtcNow;
    public DateTime? DateApplied { get; set; }
    public DateTime LastActivityDate { get; set; } = DateTime.UtcNow;

    // Relations
    public List<EmailInteraction> Interactions { get; set; } = new();
    public List<ApplicationNote> Notes { get; set; } = new();
}

public class EmailInteraction
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public Guid? JobApplicationId { get; set; }
    public JobApplication? JobApplication { get; set; }

    public string MessageId { get; set; } = string.Empty; // Deduplication key
    public string SenderEmail { get; set; } = string.Empty;
    public string Subject { get; set; } = string.Empty;
    public string BodySnippet { get; set; } = string.Empty;
    public DateTime ReceivedAt { get; set; }

    // Extracted Intelligence
    public ApplicationStatus? SuggestedStatus { get; set; }
    public bool RequiresAction { get; set; }
    public string ActionSummary { get; set; } = string.Empty;
    public DateTime? ExtractedMeetingTime { get; set; }
}
```

4. LLM STRUCTURED EXTRACTION CONTRACT

To eliminate parsing inaccuracies, the extraction engine enforces strict JSON schema definitions via the LLM API.

```
public record ExtractedEmailIntelligence(
    [property: JsonPropertyName("detected_company")] string DetectedCompany,
    [property: JsonPropertyName("detected_status")] string DetectedStatus,
    [property: JsonPropertyName("requires_user_action")] bool RequiresUserAction,
    [property: JsonPropertyName("action_summary")] string ActionSummary,
    [property: JsonPropertyName("action_items")] List<string> ActionItems,
    [property: JsonPropertyName("proposed_interview_time")] DateTime? ProposedInterviewTime,
    [property: JsonPropertyName("confidence_score")] double ConfidenceScore
);
```

5. SERVICE RESPONSIBILITIES & IMPLEMENTATION DETAILS

Service	Underlying Tech	Key Responsibilities
Email Ingestion Worker	MailKit / BackgroundService	Polls inbox folder every 5 minutes. Filters noise by domain/subject heuristic, extracts plain text body, tracks highest processed UID.
LLM Extraction Engine	Semantic Kernel / OpenAI SDK	Applies system prompt + JSON schema constraint to convert raw recruiter correspondence into structured actionable entities.
Entity Matcher Service	Fuzzy Matching / Domain Index	Correlates email sender domains (e.g. @stripe.com) or company aliases to existing JobApplication records.
Notification Dispatcher	Webhooks / OS API	Dispatches alerts (Discord, Telegram, or Windows Tray) when RequiresUserAction == true (e.g., booking links, availability requests).
Browser Extension Ingestion	Minimal API / FastEndpoints	Exposes POST /api/applications/capture to receive JDs, salary info, and URLs from the browser in 1 click.
Kanban & JD Dashboard	Blazor Server + Tailwind CSS	Provides drag-and-drop state management, interactive timelines, note taking, and searchable full-text JD archives.

6. STEP-BY-STEP IMPLEMENTATION ROADMAP

- Milestone 1 (Core Storage & Ingestion):** Scaffold .NET solution, define SQLite EF Core schema, build Minimal API endpoint, and write Chrome Manifest V3 extension.
- Milestone 2 (LLM Pipeline):** Implement IEmailIntelligenceExtractor with JSON output validation and unit test against canned recruiter emails.
- Milestone 3 (Background Worker & Matching):** Wire ImapEmailPoller with deduplication and domain-matching logic.
- Milestone 4 (Blazor UI & Alerts):** Build Kanban board interface and connect webhook notifications.